

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	S.Mahammed Nayeem	Reg. No.:	192425294
Section / Batch:	2024 - AI & ML	Date:	23-07-2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – DEBUGGING & OPTIMISATION TASK

Q1. Debugging Binary Search (Infinite Loop Issue)

The Binary Search code below leads to an infinite loop for some inputs. Identify the logical errors in updating search boundaries. Explain why the loop fails to terminate. Debug the algorithm to correctly adjust low and high indices. Optimize mid calculation if necessary. Analyze complexity and rewrite the corrected version.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid # ERROR
        else:
            high = mid # ERROR
```

Q2. Debugging Closest Pair (Wrong Initialization)

The brute-force closest pair algorithm initializes minimum distance incorrectly, leading to wrong outputs. Identify the issue and explain why results are incorrect. Debug the code to correctly initialize and update minimum

distance. Optimize to reduce redundant distance calculations. Analyze time complexity and rewrite the corrected code.

```
def closest_pair(points):
    min_dist = 0 # ERROR
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = ((points[i][0]-points[j][0])**2)**0.5
            if dist < min_dist:
                min_dist = dist
    return min_dist
```

Code of Conduct

I S.Mahammed Nayeem (192425294) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Mahammed Nayeem

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	39
Q2	4	4	4	3	2	44
Overall Total (100)						83

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Debugging Maximum–Minimum Finder Code

Aim

To debug and optimize the Maximum–Minimum Finder by handling empty input safely and removing unnecessary comparisons.

Problems in the Given Code

- No empty list check.
- Loop starts from index 0 even after initialization.
- First element is compared twice.
- Less robust and less maintainable.

Corrected Program

```
def min_max(arr):
    if not arr:
        return None

    mn = arr[0]
    mx = arr[0]

    for i in range(1, len(arr)):
        if arr[i] < mn:
            mn = arr[i]
        if arr[i] > mx:
            mx = arr[i]

    return mn, mx

numbers = [25, 10, 40, 5, 60, 18]

result = min_max(numbers)

if result:
    print("Minimum =", result[0])
    print("Maximum =", result[1])
else:
    print("List is empty.")
```

Output

Minimum = 5

Maximum = 60

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n)$
- Worst Case: $O(n)$

Space Complexity

$O(1)$

2. Debugging Linear Search Code

Aim

To debug and optimize the Linear Search algorithm by correcting the search logic.

Problems in the Given Code

- `pos` may be undefined if the element is not found.
- Unnecessary use of `found`.
- Search continues even after finding the element.
- Can produce incorrect or misleading output.

Corrected Program

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1
```

```
numbers = [12, 25, 37, 48, 59]
```

```
key = 37
```

```
position = linear_search(numbers, key)
```

```
if position != -1:  
    print("Element found at index", position)  
else:  
    print("Element not found.")
```

Output

Element found at index 2

If the element is absent (Example: key = 20)

Element not found.

Time Complexity

- Best Case: **$O(1)$**
- Average Case: **$O(n)$**
- Worst Case: **$O(n)$**

Space Complexity

$O(1)$